

New Methods for Digital Generation and Postprocessing of Random Data

Jovan Dj. Golić, *Member, IEEE*

Abstract—A new method for digital true random number generation based on asynchronous logic circuits with feedback is introduced. In particular, a concrete technique using the so-called Galois and Fibonacci ring oscillators is developed and analyzed both theoretically and experimentally. The generated random binary sequences may have a very high speed and a higher and more robust entropy rate in comparison with previous proposals for digital random number generators. A new method for digital postprocessing of random data based on irregularly clocked nonautonomous synchronous logic circuits with feedback is also introduced and a concrete technique using a self-clock-controlled linear feedback shift register is proposed. The postprocessing can provide both randomness extraction and computationally secure speed increase of input random data.

Index Terms—Random number generation, special-purpose hardware, integrated circuits, sequential circuits, chaotic systems, ring oscillators, linear feedback shift registers.



1 INTRODUCTION

IN many applications of computers and other electronic devices, there is a need for a physical source of true random numbers, for example, in computer simulations of various probabilistic algorithms, computer games, and, most notably, in possibly randomized cryptographic algorithms and protocols whose security relies on the ability to generate unpredictable secret keys and random numbers. In addition, high-speed truly random sequences are needed for countermeasures against side-channel attacks on micro-electronic devices such as smart cards. For example, such countermeasures include random masking of cryptographic functions as well as generating secret keys for the encryption of internal links and memories in such devices.

The output of a random number generator (RNG) is typically a binary sequence which is required to be unpredictable in the information-theoretic sense. Equivalently, the RNG output should ideally be statistically modeled as a purely random sequence, that is, as a sequence of mutually independent uniformly distributed binary random variables (bits), with the maximal possible entropy per bit. A necessary condition is that it should be computationally infeasible to distinguish the RNG output sequence from a purely random sequence and, in particular, to predict the RNG output sequence with a significant probability. These conditions are also necessary for pseudorandom number generators (PRNGs), which produce pseudorandom sequences from a random seed according to a state-transition function updating their internal state. A PRNG is called computationally secure if its output sequence is computationally unpredictable.

1.1 Previous Work on Digital RNGs

For practical reasons, RNGs implemented in the solid-state, semiconductor technology are preferable. A conventional type of hardware-based RNGs exploit thermal noise in resistors and/or shot noise in PN-junctions, but include analog elements and are, hence, not cost effective for many applications. Another class of RNGs exploit analog voltage-controlled oscillators (for example, see [6]). A more practical type of RNGs can be implemented by digital integrated circuits only. They are usually based on unpredictable variations in the phase and frequency (jitter) of free-running oscillators implemented as ring oscillators, where a ring oscillator consists of an odd number of inverter logic gates connected together to form a ring (for example, see [4], [3], [13], and [2]). Typically, such RNGs consist of a number of ring oscillators that are combined linearly by using an XOR operation and then sampled at a much lower speed by an independent (system) clock through a D-type flip-flop. The main disadvantages of this type of RNGs are a low speed and a tendency of the ring oscillators to couple with each other and with the system clock too, thus significantly reducing the amount of randomness produced.

Another type of digital RNGs exploits the metastability of RS latches and edge-triggered flip-flops based on RS latches such as the D-type flip-flop (for example, see [10]). Namely, the output of such a flip-flop may become unpredictable if the input and clock signals are such that the setup and/or hold times are violated, for example, if the data input signal is forced to change at nearly the same time as the clock signal. The output signal then stabilizes on a random, typically biased value after a random amount of time. The metastability of D-type flip-flops can possibly be exploited, together with the jitter of the underlying ring oscillator signals, by using D-type flip-flops for sampling the ring oscillator signals. In any case, the metastability events are relatively rare and are sensitive to temperature and voltage changes [10] so that the RNGs solely based on

• The author is with Security Innovation, Telecom Italia, Via G. Reiss Romoli 274, 10148 Turin, Italy. E-mail: jovan.golic@telecomitalia.it.

Manuscript received 26 Nov. 2004; revised 19 Oct. 2005; accepted 26 Jan. 2006; published online 22 Aug. 2006.

For information on obtaining reprints of this article, please send e-mail to: tc@computer.org, and reference IEEECS Log Number TC-0378-1104.

metastability are relatively slow and do not appear to be very reliable.

Accordingly, in spite of many (patented) proposals for hardware-based RNGs, finding an efficient and robust method for high-speed generation of true random numbers that can be implemented by using only logic gates in digital semiconductor technology still remains a challenge. The method should be efficient in terms of the gate count, achievable speed, and power consumption and should preferably be suitable for integration in both FPGA (field programmable gate array) and ASIC (application specific integrated circuit) technologies.

1.2 Previous Work on Digital Postprocessing

Typically, every true RNG essentially consists of two parts, namely, a physical source of randomness producing a raw binary sequence which is random, but not purely random, and a postprocessing part, possibly integrated in the physical source of randomness, which produces the final output sequence. For example, ring oscillator signals can be used to clock linear feedback shift registers (LFSRs) [14] or can be XOR-ed with data inputs to particular flip-flops in LFSRs [4], [3], [13] (or in linear congruential generators [11]). In the former case, the output signal produced by LFSRs should be sampled at a lower speed by the system clock, while, in the latter case, the slower system clock should be used to clock the LFSRs. In both cases, randomness is combined with pseudorandomness and, in the latter case, new randomness can possibly be introduced by effectively sampling multiple ring oscillator signals at various points in the LFSR circuit. However, as demonstrated in [5], if the ring oscillator signals are used only to clock linear circuits, then the RNG sequence may be predictable by guessing the limited phase or frequency uncertainties and by solving the linear equations.

If the raw binary sequence has a high speed, then it typically has a statistical weakness that consists of the bias of individual bits, meaning that the probabilities of the output values, 0 and 1, are not equal, and/or of the correlation of bits that are close to each other in time. The main objective of the postprocessing part is to eliminate statistical weaknesses from the raw binary sequence in the information-theoretic or only computational sense. In the first case, each output bit should approximately convey 1 bit of information or entropy and this is possible to achieve only by reducing the speed. Such postprocessing is sometimes referred to as randomness extraction. In the second case, the output sequence does not have to be purely random, but should be computationally difficult or infeasible to distinguish from a purely random sequence. In this case, the speed does not have to be reduced.

Another important objective of postprocessing is to provide robustness of the statistical properties of the RNG output sequence with respect to changes in temperature and other environmental conditions and also with respect to physical attacks on the physical source of randomness. In particular, one may also wish to deal with the case when the raw binary sequence is heavily biased. If the postprocessing is adaptive, this would then imply that the output speed varies and that it can be reduced significantly. Alternatively, one may keep the same speed, but the RNG output

sequence may then be unpredictable only in the computational rather than information-theoretic sense.

Randomness extraction techniques can be based on linear transformations only. However, they may become insecure if the speed reduction is not sufficiently large. A well-known block linear transformation is one used in the von Neumann extractor, which, together with decimation, removes the bias from a raw binary sequence provided that the input bits are statistically independent. For a generalization, see [15]. The usage of more general block linear transformations is suggested and analyzed in [1]. The technique is based on a randomly chosen and fixed linear function, which is applied to blocks of input data, taken from any RNG, to produce reduced-size blocks of output data. As the blocks have to be long, the implementation cost in terms of the gate count is considerable. By using some known results on the so-called universal hash functions, it is proven that, under certain conditions, almost all such functions give rise to nearly uniform distributions of the corresponding blocks of output data. However, one can argue that this result is not very meaningful for a fixed linear function and also that a nearly uniform distribution of output blocks does not imply a nearly uniform distribution of every (linear) transformation of output blocks. Further, this sort of randomness extraction is not sufficiently robust, namely, it does not provide computational unpredictability of output data if the entropy of input data is significantly reduced by the adversary's action on the underlying random number generator. To achieve robustness and computational security, the postprocessing technique should also incorporate nonlinear transformations.

The entropy rate of raw binary sequences generated by digital circuits usually is not high. A well-known technique for producing a high-speed (pseudo)random sequence is to apply a randomness extractor to the raw binary sequence, thus reducing its speed, and then to use this sequence to produce a random seed for a PRNG, which is then clocked at a high speed. Such a PRNG should be computationally secure and, hence, relatively complex. For example, it can be based on iterated cryptographic hash functions.

1.3 Main Objectives

The first objective of this paper is to introduce a new method and the corresponding robust techniques for generating high-speed and high-entropy raw binary sequences by using only logic gates in digital semiconductor technology. A purely random RNG output sequence can then be obtained by further postprocessing of the generated raw binary sequence. The second objective is to introduce a new method and the corresponding techniques for digital postprocessing of raw binary sequences that provide randomness extraction and computational security, as well as an increase in speed if required. Both methods should be implementable by standard digital library units and practically independent of the fabrication technology.

The main principle of the proposed general method for digital random number generation is to replace the classical ring oscillators by more complex asynchronous logic circuits with feedback and thus produce high-speed pseudorandom or even chaotic signals with more inherent randomness, due to more complex feedback signals, and

which are less susceptible to phase or frequency coupling. As the generated high-speed oscillating signal has randomness on the digital as well as analog level, the sampling D-type flip-flop may be forced into more frequent metastability conditions by using an independent sampling signal with a lower or similar frequency and this introduces more randomness. As a result, high-speed raw binary sequences with a higher entropy rate than in previously proposed purely digital RNGs can be generated.

The main principle of the proposed general method for digital postprocessing of random data is to take any computationally secure PRNG, which is typically a synchronous logic circuit with feedback implementing an autonomous finite-state machine, and operate it in a nonautonomous mode with an input taken from the physical source of randomness. In the binary case, the input can be introduced into the PRNG through XOR logic gates in a similar way as the plaintext input for stream ciphers with plaintext memory [8]. This is, in spirit, similar to the data scrambling technique used in digital communications and is a generalization of previously proposed linear techniques (see also [11]). It is proposed that the clock frequency for the PRNG may be the same as or higher than the speed of the raw binary sequence, in bits per second, and that the speed reduction, for randomness extraction, can be achieved through self-clock-controlled irregular clocking of the PRNG.

The rest of the paper is organized as follows: The general method for digital random number generation based on asynchronous logic circuits with feedback is presented in Section 2, whereas the concrete techniques introducing the so-called Fibonacci and Galois ring oscillators are proposed and analyzed in terms of the inherent randomness, pseudorandomness, and chaotic properties in Section 3. The general method for digital postprocessing of random data based on nonautonomous synchronous logic circuits with feedback is introduced in Section 4 and a concrete technique using a self-clock-controlled LFSR is described in Section 5. Conclusions and suggestions for future work are given in Section 6. Proofs of mathematical results are presented in the Appendix.

2 ASYNCHRONOUS CIRCUITS WITH FEEDBACK

A generic circuit for generating a raw oscillating signal that possesses the properties of pseudorandomness and randomness is shown in Fig. 1. A logic circuit implementing the state-transition function of an autonomous finite-state machine without fixed points is operated asynchronously, without a clock, by feeding back the outputs, representing the next state, to the respective inputs, representing the current state. In this asynchronous operation, the identity function should be implemented by a delay element, that is, by a cascade of an even number of inverters. The condition that there are no fixed points guarantees that the asynchronous logic circuit with feedback cannot get stuck in a fixed state and, thus, output a constant sequence. The state sequence is thus ensured to oscillate among a number of states. The output signal is generated by applying another logic circuit implementing the output function of the autonomous finite-state machine to the outputs of the first

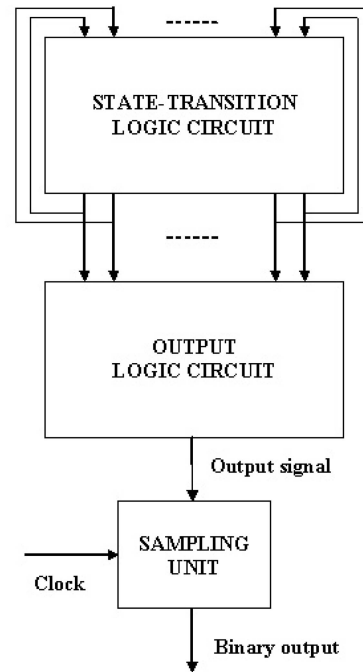


Fig. 1. A generic asynchronous circuit with feedback.

logic circuit producing the state sequence. The output function should effectively depend on the parts of the internal state that cannot get stuck at a fixed value.

The amounts of pseudorandomness and randomness contained in the oscillating signal as well as their robustness depend on the asynchronous logic circuit used. To this end, one may impose a further condition that the state-transition diagram contains only long cycles and metastable short cycles so that the state sequence cannot in practice get stuck in one of the latter. Also, the numbers of 0s and 1s per cycle should be approximately balanced.

Apart from the pseudorandomness properties according to the synchronous operation, the output signal also has randomness properties due to unpredictable variations in the delay of internal logic gates which get propagated, transformed, and enhanced through feedback, possibly in a chaotic manner, and also due to internal metastability events. The delay variations result from various internal and external noise factors possibly through unpredictable fluctuations in voltage and temperature. If the delays of internal logic gates are not long, then the output signal should have a high speed and should also exhibit randomness on the analog level as many transitions cannot be completed to the full digital level. Since the oscillating signal is then a sort of mixed digital and analog noise, further randomness due to metastability may be induced within a sampling unit, e.g., implemented as a D-type flip-flop. Its clock signal is generated by an independent ring oscillator with a lower or similar frequency as the oscillating signal. Another important feature is that the mutual coupling effect between the oscillating and sampling signals, which seriously reduces randomness in ring oscillators, is significantly reduced by the pseudorandomness of the oscillating signal.

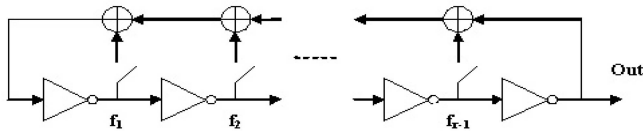


Fig. 2. Fibonacci ring oscillator.

The output sequence obtained after sampling is the desired raw binary sequence. It can have a very high speed and a high amount of entropy per bit. The final purely random output sequence can be obtained by further postprocessing to remove the residual bias and other redundancy, such as correlation, at the expense of a somewhat reduced output speed.

3 FIBONACCI AND GALOIS RING OSCILLATORS

The general principle introduced in Section 2 does not directly yield practical constructions that guarantee good randomness and pseudorandomness properties of the output signal. In the sequel, we propose two practically important concretizations that can be regarded as generalizations of the ring oscillator structure. Other concretizations, for example, based on LFSRs or programmable linear cellular automata, are also possible. Both generalizations are based on a cascade of inverters and essentially consist of replacing the simple circular feedback defining a ring oscillator by a more complex feedback incorporating a number of XOR logic gates in a way corresponding to the well-known Fibonacci or Galois configurations of an LFSR. Note that the inverters are used instead of the delay elements. The feedback connections are chosen so as to satisfy the basic condition that there are no fixed points in the corresponding state-transition function. It turns out that such feedback connections can be characterized nicely. The delays of individual inverters are the same and can be adjusted by composing each of them as a cascade of an odd number of elementary inverter logic gates. If the number of inverters is only one, then both generalizations reduce to the classical ring oscillator.

3.1 Fibonacci Ring Oscillator

The so-called Fibonacci ring oscillator, shown in Fig. 2, consists of a number, r , of inverters connected in a cascade so that the output of each but the last inverter is directly used as the input to the next inverter. The feedback connections are specified by the binary coefficients f_i with the convention that the corresponding switch is closed if $f_i = 1$ and open if $f_i = 0$, in which case, the corresponding 2-input XOR gate is not present. The feedback coefficients and the corresponding inverters are indexed from left to right. The output of the last inverter together with the outputs of the preceding inverters specified by the coefficients f_i being equal to 1 are then XOR-ed together by using, for example, 2-input XOR gates to form the feedback signal defining the input to the first inverter in the cascade. Note that the XOR operation is the same as binary addition, that is, addition modulo 2. The output signal could be taken from any inverter in the cascade.

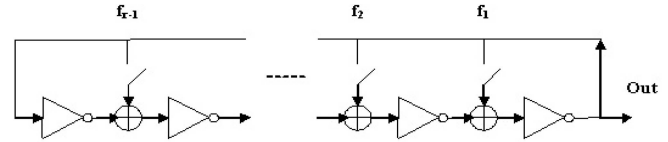


Fig. 3. Galois ring oscillator.

For both Fibonacci and Galois configurations, it is convenient to represent the feedback coefficients by a binary polynomial $f(x) = \sum_{i=0}^r f_i x^i$, $f_0 = f_r = 1$, called the feedback polynomial. (Here and throughout, the addition of binary variables is assumed to be binary.) The two following theorems are proven in the Appendix.

Theorem 1. *A Fibonacci ring oscillator does not have a fixed point if and only if*

$$f(x) = (1+x)h(x) \text{ and } h(1) = 1. \quad (1)$$

The condition (1) means that $f(x)$ is divisible by $1+x$, i.e., $f(1) = 0$, and that the quotient polynomial $h(x)$ is itself not divisible by $1+x$. It is interesting that the degree r of $f(x)$ can be odd or even, but it is necessary that $r \neq 2$.

Theorem 2. *The state-transition diagram of a synchronously operated Fibonacci ring oscillator satisfying (1) contains one short cycle of length 2, composed of the all-zero and all-one states, and a number of longer cycles whose lengths depend on the polynomial $h(x)$. If $h(x)$ is a primitive polynomial, i.e., an irreducible polynomial with the maximal period $2^{r-1} - 1$, then there is only one long cycle of length $2^r - 2$.*

A primitive polynomial $h(x)$ is recommended to ensure good pseudorandomness properties of the output signal. It follows that the short cycle is metastable in the asynchronous operation for $r > 1$, which means that, in practice, the state sequence can spend only a very short time in this cycle and then leaves the cycle by reaching an unpredictable state. The transition between these two states is metastable because, if the outputs of all the inverters quasi-simultaneously reach an intermediate voltage value, halfway between the voltage values representing the binary states 0 and 1, then a further transition in the same direction is no longer sustained and, hence, becomes unpredictable.

3.2 Galois Ring Oscillator

The so-called Galois ring oscillator, shown in Fig. 3, consists of a number, r , of inverters connected in a cascade so that the output of each but the last inverter is used to form the input to the next inverter and the output of the last inverter directly defines the feedback signal. The feedback connections are specified by the binary coefficients f_i with the convention that the corresponding switch is closed if $f_i = 1$ and open if $f_i = 0$, in which case, the corresponding 2-input XOR gate is not present. The feedback coefficients and the corresponding inverters are indexed from right to left. The input to the first inverter in the cascade is directly defined by the feedback signal. If $f_i = 0$, then the input to the i th inverter is directly defined by the output of the $(i+1)$ th inverter. If $f_i = 1$, then the input to the i th inverter is formed by XOR-ing the output of the $(i+1)$ th inverter

with the feedback signal. In Fig. 3, the output signal is taken from the last inverter in the cascade.

The two following theorems are proven in the Appendix.

Theorem 3. *A Galois ring oscillator does not have a fixed point if and only if*

$$f(1) = 0 \text{ and } r \text{ is odd.} \quad (2)$$

The condition (2) means that $f(x) = (1 + x)h(x)$ and that the degree of $f(x)$ is odd.

Theorem 4. *The state-transition diagram of a synchronously operated Galois ring oscillator satisfying (2) contains one short cycle of length 2 if and only if an additional condition $h(1) = 1$ is satisfied. The number and lengths of longer cycles then depend on the polynomial $h(x)$. If $h(x)$ is a primitive polynomial, then the additional condition is satisfied and there is only one long cycle of length $2^r - 2$.*

A primitive polynomial $h(x)$ is recommended to ensure good pseudorandomness properties of the output signal. The short cycle of length 2 is composed of two states of the following form: Divide the r inverters into $2n + 1$ groups separated by the effective feedback connections, specified by nonzero feedback coefficients, and let, as before, the inverter output bits define the state vector. Then, the two states have the forms $c_{n+1}^* A_n c_n^* A_{n-1} \cdots A_2 c_2^* A_1 c_1^*$ and $\overline{c_{n+1}^*} A_n \overline{c_n^*} A_{n-1} \cdots A_2 \overline{c_2^*} A_1 \overline{c_1^*}$, respectively, where A_i is an alternating string, c_i^* is a constant string obtained by repeating a constant c_i , and $\overline{c_i^*}$ is a constant string defined by the binary complement $\overline{c_i}$. All the strings are uniquely determined by the following rule: If $c_1 = 0$, then the last bit of A_i equals c_i and the first bit of A_i equals $\overline{c_{i-1}}$. For example, for $f(x) = 1 + x^2 + x^6 + x^7$, the two states are 0101000 and 1101011.

It thus follows that the transition between these two states is metastable in the asynchronous operation for $r > 1$ unless, maybe, if all the constant strings have the same odd length, for example, equal to one. Namely, a stable transition would first imply that there is a stable oscillation between each constant string and its binary complement and, second, that none of the alternating strings changes in time. The former is extremely unlikely to happen unless all the constant strings have length one, for reasons similar to those already explained in the context of the Fibonacci configuration. For the latter to happen, it is necessary that all the constant strings change practically simultaneously and this is extremely unlikely unless, maybe, if all the constant strings have the same odd length. If the length of a constant string is even, then a transition between this string and its binary complement gives rise to a metastability event of the same type as that occurring in an RS latch (due to an even number of inverters with the same output value connected in a loop), which causes the metastability of a D-type flip-flop. In particular, since the output signal is taken from the last inverter in the cascade, it is recommended that the length of the last constant string, c_1^* , should be even. For example, this length is equal to two if $f_1 = 0$ and $f_2 = 1$.

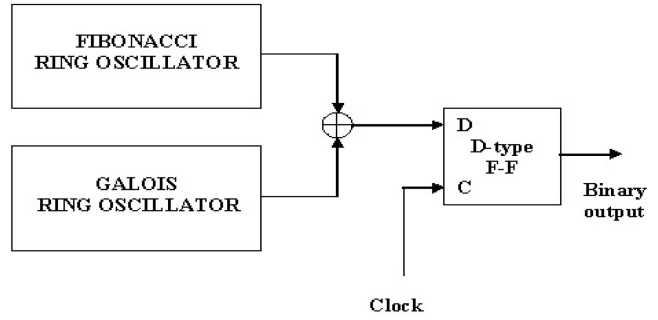


Fig. 4. An elementary digital RNG.

3.3 Combined Oscillator

The randomness as well as robustness can be further increased by XOR-ing the outputs of two oscillators, one being in the Galois and the other in the Fibonacci configuration. This also combines the properties of the output signals of the two configurations. The corresponding elementary RNG, including a sampling unit implemented as a D-type flip-flop, is shown in Fig. 4. The lengths of the two oscillators minus one should preferably be mutually prime, first, for the period of the corresponding pseudorandom sequence to be maximized and, second, for the interlocking or coupling effect to be minimized. In particular, it is suggested that the lengths differ only by one, where the even length corresponds to the Fibonacci ring oscillator. In practice, the lengths could be around 20.

3.4 Frequency, Pseudorandomness, and Output Period

For both configurations, the output oscillating signal is irregular and possesses both pseudorandomness and randomness properties. In a classical ring oscillator, the average frequency is the reciprocal of the average period, $2\Delta t$, where Δt is the average delay of the inverter used. For an irregular oscillating signal, the average frequency can be defined as the reciprocal of the average time between two successive rising edges (i.e., transitions from 0 to 1) in the signal. If the oscillating signal is modeled as a random binary signal with the average duration time of a single bit equal to ΔT , then the average frequency is $1/(4\Delta T)$.

In a Galois or Fibonacci ring oscillator, ΔT is the greatest common divisor of the average inverter delays of all the feedback loops. For r inverters with the average inverter delay Δt , if the feedback delays are neglected, then $\Delta T = d\Delta t$, where $d = \gcd(i : f_i = 1, 1 \leq i \leq r)$. As the feedback polynomial then has the form $f(x) = g(x^d)$, the oscillator can be reduced to an equivalent oscillator with the feedback polynomial g and the inverter delay increased d times. It is hence desirable that $d = 1$. In this case, $\Delta T = \Delta t = k\Delta \tau$, where $\Delta \tau$ is the average delay of an elementary inverter logic gate and k is an odd number of elementary inverter logic gates comprising each inverter. For a high-frequency oscillating signal, it is thus required that $d = 1$ and that k is small. Note that the conditions $f(x) = (1 + x)h(x)$ and $h(1) = 1$ imply that d is odd and if, in addition, $h(x)$ is irreducible, then $d = 1$.

The power consumption is proportional to the product of the total number of logic gates and the average switching

frequency at which they change their states, which is two times higher than the average frequency specified above. Since this frequency is inversely proportional to the number of inverter logic gates per inverter, the product roughly depends only on the number of inverters if the feedback logic gates are not counted. The same is true for classical ring oscillators, but then there is typically a lower bound on the number of inverter logic gates that guarantees a sufficient gain factor for stable oscillations. As in the new oscillators the gain factor is provided by $r > 1$ inverters, the frequency can be higher.

In the basic analysis of pseudorandomness properties of the oscillating signal, it is assumed that all elementary inverter logic gates have the same fixed delay, $\Delta\tau$, that the feedback logic gates have zero delay, and that each logic gate has zero transition time. It is also assumed that the inverter outputs change synchronously. In this model, the oscillating signal is binary and pseudorandom according to the synchronous mode of operation in which each inverter is replaced by a synchronously clocked D-type flip-flop followed by an inverter logic gate. Under the conditions specified in Sections 3.1 and 3.2, with r inverters and k inverter logic gates per inverter, the period of the oscillating signal is then equal to $(2^r - 2)k\Delta\tau$ and its frequency equals $1/(4k\Delta\tau)$. Note that the expression for the period also formally holds for a classical ring oscillator by setting $r = 2$. If the sampling ring oscillator is composed of k_0 inverter logic gates, with the same delay $\Delta\tau$, then its period is equal to $2k_0\Delta\tau$. If the sampling takes place at the rising or falling edge of the sampling oscillator signal, then the period P of the output binary sequence satisfies

$$P \mid \frac{k(2^{r-1} - 1)}{\gcd(k_0, k(2^{r-1} - 1))}. \quad (3)$$

In order to avoid correlations of successive bits in the output binary sequence due to sampling the same binary signal value, a natural requirement is that $2k_0 \geq k$.

For a combined ring oscillator from Section 3.3, we can assume that the underlying r_1 and r_2 inverters in the Galois and Fibonacci ring oscillators each consist of k_1 and k_2 inverter logic gates, respectively, where k_1 and k_2 can be different. The period of the oscillating signal then divides the least common multiple of the periods of the two oscillators, that is, $2\Delta\tau \text{lcm}(k_1(2^{r_1-1} - 1), k_2(2^{r_2-1} - 1))$, and its frequency approximately increases to the reciprocal of $4\gcd(k_1, k_2)\Delta\tau$. The period P of the output binary sequence then satisfies

$$P \mid \frac{\text{lcm}(k_1(2^{r_1-1} - 1), k_2(2^{r_2-1} - 1))}{\gcd(k_0, \text{lcm}(k_1(2^{r_1-1} - 1), k_2(2^{r_2-1} - 1)))} \quad (4)$$

and it is required that $2k_0 \geq \gcd(k_1, k_2)$.

3.5 Enhanced Randomness

In the basic model from Section 3.4, the binary pseudorandomness provided by an appropriate choice of the feedback polynomial(s) can be considered as a sort of digital chaos with respect to the inverter delay as the basic time unit for changes in the binary output signal.

In a more realistic model for analyzing the pseudorandomness, one can also take into account delays of the

feedback logic gates as well as possible differences in the delays of inverter logic gates. Note that the delays of the feedback gates are more influential in the Fibonacci configuration, especially if the inverter delay is relatively small. For the analysis to be tractable, one can assume that all the logic gates involved have commensurate delays, that is, delays being integer multiples of a unit delay, which is their greatest common divisor. In the associated synchronous mode of operation, a number of synchronously clocked D-type flip-flops are added in cascade at the output of each logic gate, where this number is equal to the corresponding integer specifying the logic gate delay. Pseudorandomness properties then correspond to the state-transition function of the resulting finite-state machine. If the feedback logic gates are XOR gates, then the finite-state machine is linear and, as such, mathematically analyzable. In the case of Galois and Fibonacci ring oscillators, the associated linear finite-state machine effectively has a modified feedback polynomial with an increased degree and the changes in the output signal correspond to the unit delay. The modified feedback polynomial depends on a particular hardware implementation. For example, in the Fibonacci configuration, apart from the XOR gate delays, the modified feedback polynomial also depends on how the XOR gates are grouped together to form the feedback signal.

In this model, where all the logic gates contained have fixed delays and zero transition times, the unit delay with which all the logic gate delays are commensurate can in theory be much smaller than the average delay of an inverter logic gate. The binary output signal then changes with the unit delay as the time unit, but these changes may be slow and then cannot be regarded as pseudorandom with respect to this time unit. In this case, the average frequency is smaller than what one may expect from a random binary signal with the average duration time of a single bit equal to this time unit. However, if the changes in the binary output signal occur rapidly and pseudorandomly, with the unit delay as the time unit, then the output signal may exhibit binary pseudorandomness with a very long period which can be considered as a sort of digital chaos with respect to this time unit. This chaotic behavior is not present in classical ring oscillators and, in general, may depend on other properties of the feedback polynomial(s) such as the degree and the number of feedback taps.

The model above is only an approximation as the logic gates have nonzero transition times and delays that both vary in time. Due to nonzero transition times, the changes between the binary values are gradual rather than abrupt so that the oscillating signal also contains analog parts corresponding to the logic gate transitions, the more so if the inverter delay is relatively small so that the delay-to-rise time ratio is also relatively small. Also, as there is not enough time for very short binary pulses resulting from the model described above to be completed, this gives rise to very short analog pulses, with varying amplitude. As a consequence, even if we assume that the delays and transition times of all the logic gates are fixed, the output oscillating signal may have a very high frequency and may exhibit pseudorandomness on the analog as well as digital

level, which can be regarded as a sort of chaotic behavior. This chaotic phenomenon does not exist in classical ring oscillators.

The variations of the delays and transition times of the logic gates are commonly considered as random and are due to various internal and external noise factors such as thermal noise and unpredictable short-term or long-term fluctuations in voltage and temperature. As such, the underlying randomness is influenced by a particular hardware implementation. Random delays and transition times result in an irregular oscillating signal having both randomness and pseudorandomness features on the binary as well as analog level. Each random variation is propagated through feedback and may, hence, result in a chaotic change of the subsequent oscillating signal. All such changes are combined together to produce randomness in the oscillating signal, which appears to be difficult to quantify.

Our next objective is to compare classical ring oscillators with Galois or Fibonacci ring oscillators with respect to the amount of randomness produced. The first two remarks pertain to the oscillating signals, while the last three remarks pertain to the output binary sequences obtained after sampling.

Increased primary randomness. The primary sources of randomness are random delays and transition times of all the logic gates in the circuit, and the amount of randomness generated per time unit can thus be measured by the product of the total number of logic gates and the average switching frequency. This product is proportional to the power consumption and is roughly dependent only on the number of inverters if the contribution of the feedback XOR logic gates is neglected (see Section 3.4). As in a Galois or Fibonacci ring oscillator, in comparison with a classical ring oscillator, this number is increased r times, while the average switching frequency is two times lower, the amount of primary randomness is thus increased at least about $r/2$ times. This amount further increases as the switching frequency is higher due to the chaotic behavior of the oscillating signal. In addition, the irregular oscillating signal may itself result in larger variances of the random delays of individual logic gates.

Internal metastability. During the oscillations, additional randomness may be generated due to internal metastability events resulting from the feedback loops involving chains of inverters. The reasons are similar to those behind the metastability of the short cycle states explained in Sections 3.1 and 3.2. The metastability event will occur at a time when the outputs of all the inverters on a feedback loop as well as the input to the first inverter on this loop quasi-simultaneously reach an intermediate voltage value and when the input signal to the loop, coming from the remaining inverters through an XOR logic gate, has a constant value such that the change of the feedback signal does not sustain the change of the input signal to the first inverter, which, therefore, remains at the intermediate voltage value together with the other inverters on the loop, thus giving rise to metastability. This phenomenon is not present in classical ring oscillators for $r = 1$.

Reduced mutual coupling. In a classical ring oscillator, the sampling and oscillating signals are both regular and periodic with a short period. As such, they tend to couple with each other with respect to phase and frequency, which reduces the amount of randomness in the output binary sequence. In a Galois or Fibonacci ring oscillator or their combination, the oscillating signals are irregular, pseudorandom with a long period, and possibly chaotic. This significantly reduces their mutual coupling and coupling with the regular sampling signal.

Amplified randomness via sampling. The chaotic behavior of the oscillating signal and increased primary randomness give rise to a frequency with a higher average value and a larger standard deviation. The randomness is thus transformed into a form more suitable for extraction by sampling. The sensitivity to and the amount of relative phase jitter are then both increased. Consequently, the sampling frequency can be made higher without reducing the entropy per bit in the sampled sequence and this yields a higher entropy rate. Note that increasing the sampling frequency may introduce correlations between successive bits if the randomness accumulated during one sampling period is insufficient.

More frequent sampling metastability. In the sampling circuit, such as an edge-triggered D-type flip-flop, metastability may occur if the setup and/or hold times are violated during sampling. Due to the irregular high-frequency oscillating signal, which is pseudorandom and possibly chaotic on the binary as well as analog level, the metastability events are more frequent in a Galois or Fibonacci ring oscillator or their combination than in a classical ring oscillator. The metastability typically results in slightly biased output bits, but the corresponding reduction in entropy is very small. Also, as very short analog pulses in the input signal may occasionally not be sufficient to change the output, the output bits may be slightly correlated. In order to enhance the metastability, dedicated sampling circuits may be designed to make better use of the irregular noise-like waveforms of the oscillating signal.

3.6 Experimental Results

In order to verify the ability of the new oscillators to generate high-speed oscillating signals with inherent randomness, pseudorandomness, and chaotic properties as well as high-speed and high-entropy raw binary sequences after sampling, we conducted experiments in FPGA technology (XILINX xc2v3000-4) by using a logic analyzer and an oscilloscope. A library D-type flip-flop was used for sampling and, in each oscillator, at least one of the inverter logic gates was implemented by a NAND gate. It is important to emphasize that, in FPGA technology, we did not have any control over the layout of logic gates.

In the experiments, we examined Galois and Fibonacci ring oscillators as well as their XOR combinations for various lengths r , feedback polynomials, numbers of inverter logic gates per inverter, and numbers of inverter logic gates in the sampling ring oscillator. The raw binary sequence was segmented into successive 32-bit blocks which were sequentially stored in a buffer register and then supplied in parallel to the input of the logic analyzer, which is capable of producing histograms for up to 689 groups of 32-bit values

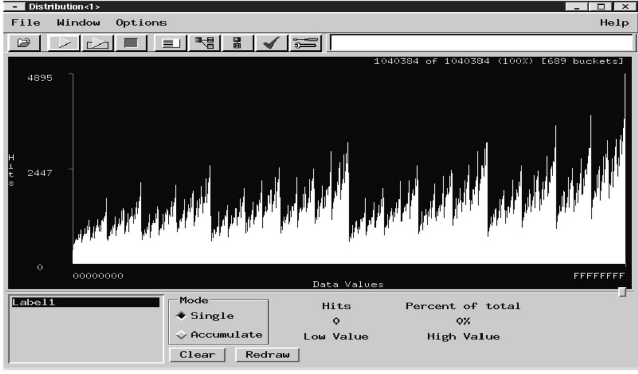


Fig. 5. A saw type of histograms.

estimated on about one million inputs. In particular, histograms can be produced for any subset of up to 9 bits of 32 bits and can be viewed either cumulatively or one after another in time. It turns out that they conveniently reflect the statistical properties of the raw binary sequence. The decimation by 32 cannot reduce the odd period given by (3) or (4).

In each case, we observed histogram pictures on the logic analyzer and, in some cases, we also observed pictures on the oscilloscope of the underlying oscillating signal before sampling. Note that the introduction of the oscilloscope requires a change in hardware implementation and, hence, may influence the oscillating signal. The observed histogram pictures can be roughly classified into two main types, namely, the saw type and the line type, where the line type is rare. The saw type is very similar to the histograms corresponding to a bias of individual, mutually independent bits and is, hence, asymmetric and increases to the right or to the left depending on whether the bias is toward the value of 1 or 0, respectively. The saw type can be relatively smooth or relatively spiky, depending on how large the bias is. For illustration, a moderately spiky saw type is displayed in Fig. 5. The line type consists of isolated lines, which means that some of the 32-bit values never appear. The line type can be relatively sparse or relatively dense. In a very small number of cases, we also observed instability in that the saw and the line types were randomly interchanged in time.

It is interesting that the same saw type of histograms can be observed for the binary sequences obtained at the output of a sampling D-type flip-flop, with the input and clock signals supplied by two classical ring oscillators, after detecting the occurrence of metastability events by an appropriate circuit [7]. The detected metastability events for this structure were rare (say, less than 1 in 100) and result in biased and roughly independent output bits. The similarity of histograms is an indication that the sampling metastability may be an important additional source of randomness in the new oscillators, where there is no metastability detection and every sampled bit is taken to the output.

In the experiments with classical ring oscillators, we always obtained a sparse and concentrated line type of histograms, even for relatively large sampling periods (that is, k_0). For ASIC implementations, it is commonly estimated that, for $k_0/k \geq 30$, the output bits may become roughly

unbiased and independent so that the resulting histogram should be relatively flat. In our experiments, the line type of histograms became somewhat denser by increasing k_0 , but still remained relatively sparse even for $k_0/k \geq 100$. This is probably due to the FPGA implementation and may indicate a limited variance of inherent random delays and a strong mutual coupling effect. Note that, for fixed delays of inverter logic gates, in view of (3), the period of the output binary sequence should divide $k/\gcd(k_0, k)$, which itself results in a sparse and concentrated line type of histograms.

We did experiments for different lengths r , e.g., $r = 7, 13, 20, 21, 33$. In some cases, we examined the oscillating signals on the oscilloscope and the observed pictures correspond to mixed binary and analog noise-like signals as explained in Section 3.5. The line type of histograms was rare and was observed for individual Galois or Fibonacci ring oscillators, for some k_0 and for both small and larger r . They may possibly result from a sort of mutual coupling. For the XOR combinations of Galois and Fibonacci ring oscillators, the saw type of histograms was generally smoother, as should be expected. We examined the histograms by increasing the sampling period and observed that, sometimes, they became smoother, sometimes not, but they never became practically flat, at least for the values of k_0 examined. Interestingly, by varying k_0 , in many cases, the type of bias also varied, toward 0 or 1. If the properties of the oscillating signal do not change when k_0 is varied, then the bias of the sampled signal cannot change considerably, but only the correlation between the successive bits. The experiments thus show that the properties of the oscillating signal change or the bias induced by metastability in the sampling D-type flip-flop changes and both situations are possible since the FPGA configuration changes when k_0 is varied. The bias itself may be due to relatively frequent sampling metastability or to the inherent bias in the oscillating signal. As the bias in the oscillating signal due to pseudorandomness on the inverter delay level as the time unit is impossible, this may indicate pseudorandomness on shorter time units due to the chaotic behavior explained in Section 3.5.

For example, the relatively smooth or moderately spiky saw type of histograms, with a typical bias ratio around 0.4/0.6, were obtained in all the cases if the feedback polynomials for the Fibonacci and Galois configurations were $1 + x^2 + x^4 + x^5 + x^{12} + x^{13} + x^{15} + x^{16} + x^{18} + x^{20}$ and $1 + x^2 + x^3 + x^7 + x^{16} + x^{17} + x^{19} + x^{21}$, with five and three inverter logic gates per inverter, respectively, and with 11 inverter logic gates in the sampling ring oscillator. The resulting speed was about 100 Mbps.

In order to better differentiate between pseudorandomness on the inverter delay level and randomness or pseudorandomness on shorter time units due to the chaotic behavior, we examined relatively short oscillators, e.g., $r = 7$. For example, we experimented with the feedback polynomials $1 + x^2 + x^6 + x^7$ and $1 + x + x^5 + x^7$ for individual Galois or Fibonacci configurations or for their XOR combination. In this case, for fixed delays of inverter logic gates, if $k_1 = k_2 = k$ and $k_0 = kn_0$, then, in view of (3) and

(4), the period, P , of the output binary sequence satisfies $P \mid 63/\gcd(n_0, 63)$. This itself would result in a line type of histograms in the absence of randomness and chaotic behavior. In some cases, we indeed observed the line type or unstable mixed saw and line types, especially for individual Galois or Fibonacci ring oscillators.

However, in most cases, we observed the saw type, which was relatively smooth in many cases, including the values $n_0 = 3, 5, 7, 21, 99$ when the period is expected to be only $P \mid 21, 63, 9, 3, 7$, respectively. For example, among such cases, we mention the Galois and Fibonacci ring oscillators with the feedback polynomial $1 + x + x^5 + x^7$, $k = 7$, and $n_0 = 3$ and $n_0 = 7$, respectively, as well as the XOR combinations of the Galois and Fibonacci ring oscillators with the same feedback polynomial $1 + x^2 + x^6 + x^7$ and $k = 3$, for $n_0 = 3$ and $n_0 = 21$. This clearly shows the presence of randomness or chaotic properties in the oscillating signal and the corresponding raw binary sequence which cannot be found in classical ring oscillators. On the other hand, experimental differentiation between randomness and chaotic properties as well as among different sources of randomness described in Section 3.5 appear to be difficult problems on their own which are out of the scope of this paper.

The detected type of redundancy present in the raw binary sequence, namely, a slight bias of individual bits and a slight correlation of adjacent bits, can be easily removed by digital postprocessing that reduces the output speed by a small factor.

4 SYNCHRONOUS NONAUTONOMOUS CIRCUITS WITH FEEDBACK FOR POSTPROCESSING OF RANDOM DATA

A generic circuit for digital postprocessing of raw random data is shown in Fig. 6. The raw binary sequence generated at the output of an RNG is used as the input sequence to a PRNG which is implemented as a synchronous nonautonomous finite-state machine. It is described in terms of the state-transition and output functions and operates synchronously with a clock signal. The state-transition and the output functions should be chosen so as to satisfy the *computational unpredictability criterion*: It should be difficult (ideally, computationally infeasible) to predict the PRNG output sequence when the input is the all-zero sequence or, more generally, when the input sequence is fixed. This implies that the PRNG internal state bit size should not be small. The state-transition function can be implemented as a composition of a logic circuit and a parallel combination of D-type flip-flops, used as delay elements, whereas the output function is implemented by another logic circuit.

The input sequence is introduced through an XOR operation, which, at each time, combines an input bit with one or more internal state bits, which are then fed back into the logic circuit along with other internal state bits to produce the internal state at the next time. For simplicity, only one such internal state bit is shown in Fig. 6. These internal state bits are chosen so as to satisfy the *propagation criterion*: For any initial state, a change of the first input bit should give rise to a pseudorandom change of the

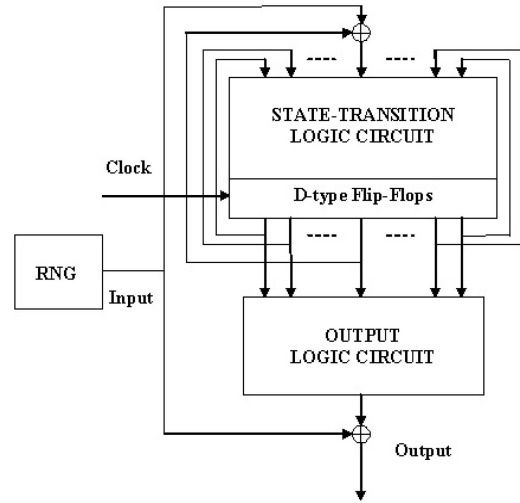


Fig. 6. A generic postprocessing circuit.

subsequent output bits, namely, to a change difficult (ideally, computationally infeasible) to predict. The output function, depending on the current internal state bits, is combined with the current input bit to produce the output bit so as to satisfy the *randomness criterion*: The output sequence should be purely random if the input sequence is purely random. To this end, it is sufficient to combine the current input bit and the output function by another XOR operation, as shown in Fig. 6. This corresponds to a technique for designing stream ciphers with plaintext memory [8]. The PRNG initial state can be fixed or (pseudo)randomly chosen.

The described generic circuit implements a reversible sequential transform, provided that the initial state is known. This implies that any statistical distinguisher for the input sequence can be converted into a distinguisher for the output sequence by guessing the internal state at any time and recovering the input sequence. Consequently, an important component of the generic circuit is irregular clocking, described below.

The clock signal for triggering the D-type flip-flops should have the same or higher frequency as the speed of the input sequence, in bits per second. It can be synchronous with a clock used in the RNG to produce the raw binary sequence. The reduction in speed is proposed to be achieved by irregular clocking in the self-clock-controlled manner according to the clock-control sequence produced by applying another function to the internal state sequence. This function should be chosen so as not to violate the computational unpredictability criterion and, for simplicity, is not shown in Fig. 6. If the clock-control sequence is binary, then, at each time when an output bit is produced, the value of the clock-control bit determines the number of times, δ_1 or δ_2 , the D-type flip-flops should be clocked before the next output bit is produced. If the clock-control bit is not biased, then the expected reduction in speed due to irregular clocking is $(\delta_1 + \delta_2)/2$ times. Note that, if $\delta_1 = \delta_2$ and if the PRNG is linear, then it may be possible to recover linear transforms of the input sequence from the resulting uniformly decimated output sequence. This in turn may give rise to statistical distinguishers for the output

sequence, but the deviation from the uniform distribution is significantly reduced as linear transforms of the input sequence are expected to involve a large number of terms unless, maybe, if $\delta_1 = \delta_2 = 1$.

5 SELF-CLOCK-CONTROLLED LFSRS FOR POSTPROCESSING

A concretization of the generic circuit from Section 4 that is suitable for hardware implementations is proposed in the sequel. Essentially, it is a self-clock-controlled LFSR in the Galois configuration that is operated nonautonomously. Such an LFSR is a cascade of D-type flip-flops where the output of the last flip-flop defines the feedback signal that is used as input to the first flip-flop and is XOR-ed with outputs of some flip-flops to form the inputs to the succeeding flip-flops in the cascade. It is well-known that binary sequences with a long period and good statistical properties can be produced if the feedback connections are chosen according to a primitive or irreducible polynomial.

A regularly clocked LFSR is a linear sequential circuit whose output is easily predictable by solving the corresponding linear equations. The linear predictability can be destroyed if an LFSR is clocked irregularly, that is, if some of its output bits are discarded according to a clock-control signal. So, in order to define a computationally secure PRNG, one can use one or more LFSRs, possibly irregularly clocked, and combine their outputs by a suitable function. Such LFSR-based PRNGs are very suitable for hardware implementations.

The simplest PRNG of this type contains a single self-clock-controlled LFSR in the Galois or Fibonacci configuration. For the Galois configuration, the basic structure of the corresponding postprocessing circuit is shown in Fig. 7. The LFSR consists of a number, r , of D-type flip-flops connected in a cascade so that the output of each but the last flip-flop is used to form the input to the next flip-flop and the output of the last flip-flop directly defines the feedback signal. All the flip-flops are clocked synchronously by the same clock signal. The feedback signal is XOR-ed into the flip-flops in an analogous way as in the Galos ring oscillator described in Section 3. In order to ensure the maximal period, $2^r - 1$, and good statistical properties, it is recommended to choose a primitive binary feedback polynomial.

The LFSR operates in the nonautonomous manner by XOR-ing the input signal with the feedback signal. The output is formed by XOR-ing the output of the XOR gate connected to the output of the γ th flip-flop, where $f_{\gamma-1} = 1$, with the output of the 2-input OR gate applied to the outputs of the j th and k th flip-flops. Formally, if one takes $\gamma = 1$, then the output is formed by XOR-ing the feedback signal with the output of the OR gate.

The clock-control sequence is binary so that, at each time when an output bit is produced, the value of the clock-control bit determines the number of times, δ_1 or δ_2 , the LFSR should be clocked in order to produce the next output bit. The clock control can be implemented in hardware by a counting logic and another D-type flip-flop by using its clock-enable input. The clock-control bit is produced by XOR-ing the outputs of the τ_1 th and τ_2 th flip-flops. It is

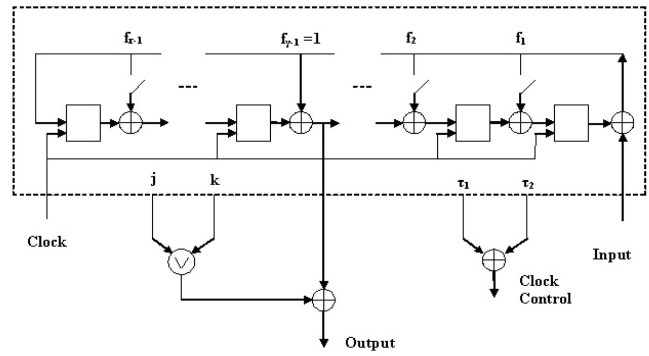


Fig. 7. An LFSR-based postprocessing circuit.

suggested that $\min(j, k, \gamma) > \max(\tau_1, \tau_2)$. Since the clock-control bit is not biased, the expected speed of the output sequence is $(\delta_1 + \delta_2)/2$ times smaller than the clock frequency.

The security of the proposed PRNG depends on the LFSR length r and, for the RNG postprocessing applications, it is suggested that $64 \leq r \leq 128$ may be sufficient. Also, the security is affected by the number of nonzero feedback coefficients, that is, by the weight of the feedback polynomial, which is suggested to be close to $r/2$. In addition, the weight of low-degree polynomial multiples of the feedback polynomial should not be small. In particular, these conditions should practically prevent the types of statistical distinguishers described in [9], even if the input sequence is heavily biased. The security level can be further increased at the expense of an increased gate count by using a number of LFSRs that are irregularly clocked in the self-clock-control or mutual-clock-control manner.

The postprocessing circuit from Fig. 7 was implemented in FPGA technology together with the RNG circuit from Fig. 4 and the produced binary sequences were subjected to statistical tests. For example, the choice of $r = 64$ in the postprocessing circuit was sufficient to satisfy the DIEHARD suite of tests [12] and standard statistical tests for binary sequences such as the frequency, serial, poker, autocorrelation, runs, and Maurer tests. This was the case even for the all-zero input sequence and $\delta_1 = \delta_2 = 1$, but then, in order to satisfy the linear complexity test, we have to use irregular clocking, that is, $\delta_1 \neq \delta_2$.

In practice, the choice of $\delta_1 = 3$ and $\delta_2 = 5$ may be appropriate. In this case, the tests are also satisfied for relatively small r (e.g., at most 30), but then are not satisfied for the all-zero input sequence. The average reduction in speed of about four times is heuristically suggested according to the following experiment. Namely, for a simplified postprocessing circuit in which $r = 1$ and $\delta_1 = \delta_2 = 4$, which corresponds to the cumulative XOR sum of input bits, the tests were satisfied for most RNG configurations chosen. Of course, any further reduction in speed can only result in higher entropies per bit achieved.

6 CONCLUSIONS

A new method for true random number generation by using digital circuits only is introduced and preliminarily experimentally tested in FPGA technology. The method is

robust and may achieve much higher entropy rates than other known digital techniques. The main point is to replace the free-running ring oscillators by new oscillators generating high-speed pseudorandom and chaotic signals with enhanced randomness properties on the digital as well as analog level and to sample these signals by an independent ring oscillator signal with a lower or similar frequency possibly causing more frequent metastability events in the sampling D-type flip-flop and significantly reducing the mutual coupling effect.

The method is supported by an in-depth mathematical analysis of the underlying synchronous mode of operation. Also, the asynchronous mode of operation is analyzed in terms of frequency and output period as well as pseudorandomness and chaotic properties, and a more detailed analysis merits further research. A number of sources of enhanced true randomness, including increased primary randomness, internal metastability, reduced mutual coupling, amplified randomness via sampling, and more frequent sampling metastability, are pointed out. The combined sources of randomness provide robustness, whereas their combination with pseudorandomness and chaotic properties also provides computational unpredictability if the amount of randomness is reduced. Both the analysis and experimental results thus clearly indicate the advantages of the new oscillators in comparison with ring oscillators. As the resulting increase in entropy rate appears difficult to quantify and may depend on a particular implementation, entropy estimation is an interesting problem to deal with and may be complicated by a chaotic behavior. A more systematic experimental analysis, in FPGA or ASIC technologies, to better differentiate between randomness and chaotic properties is another area for future investigations, especially for high-security applications.

A new method for digital postprocessing of raw random data which can provide both randomness extraction and computationally secure increase in speed is also introduced. The method is suitable for hardware implementations.

The methods proposed are interesting for any application of computers and other electronic devices needing a physical source of true random numbers. In particular, they are interesting for applications in cryptographic algorithms and protocols, not only for generating secret keys and random numbers such as random nonces, but also for achieving robust implementations, resistant to side-channel and other implementation-based attacks.

APPENDIX

MATHEMATICAL PROOFS

Proof of Theorem 1. The state vector consists of binary outputs of individual inverters in the cascade, taken in order of increasing indexes. We first characterize a Fibonacci ring oscillator that has at least one fixed point. According to the corresponding state-transition function, a state vector is a fixed point only if it is an alternating string of 0s and 1s, that is, only if it is equal to 0101... or 1010... The vector 0101... is a fixed point if and only if the corresponding feedback value is equal to 1, that is, if and only if $f_2 + f_4 + \dots = 1$, where the sum is over even

indexes. Similarly, the vector 1010... is a fixed point if and only if the corresponding feedback value is equal to 0, that is, if and only if $f_1 + f_3 + \dots = 0$, where the sum is over odd indexes.

Consequently, a Fibonacci ring oscillator does not have a fixed point if and only if neither 0101... nor 1010... is a fixed point, that is, if and only if $f_2 + f_4 + \dots = 0$ and $f_1 + f_3 + \dots = 1$. The two conditions are equivalent to $f(1) = 0$ and $f_1 + f_3 + \dots = 1$. Further, $f(1) = 0$ is equivalent to $f(x)$ being divisible by $1 + x$, i.e., to $f(x)$ having the form $f(x) = (1 + x)h(x)$. As for $f_1 + f_3 + \dots = 1$, we use $h(1) = f_1 + f_3 + \dots$, which directly follows from $f(x) = (1 + x)h(x)$. This means that $f_1 + f_3 + \dots = 1$ is equivalent to $h(1) = 1$, provided that the first condition, $f(1) = 0$, is satisfied. $\square$

Proof of Theorem 2. In the synchronous operation, the inverters are implemented as delay elements (e.g., edge-triggered D-type flip-flops), synchronously clocked by the same clock signal, whose outputs are complemented by inverter logic gates or, equivalently, by adding the constant 1 by using 2-input XOR gates, respectively. Consequently, each new clock gives rise to a synchronous change of the internal state, according to the underlying state-transition function, and a state sequence is thus produced starting from any given initial state. The corresponding finite-state machine is a nonautonomous LFSR with the all-one sequence as its input, which is XOR-ed in at the output of each delay element. As this finite-state machine is linear, a mathematical expression for the state sequence, as a function of a given initial state, can be obtained by using the D-transform (i.e., generating function) technique, well known in the theory of LFSRs. The D-transform of a binary sequence $y = (y_t)_{t=0}^{\infty}$ is defined as a formal power series $Y = \sum_{t=0}^{\infty} y_t D^t$. Here, D can be regarded as a delay operator since DY is the D-transform of a delayed sequence y , i.e., of y phase-shifted to the right by one time instant, with the appended zero initial bit, for $t = 0$.

As r binary component sequences of the state sequence, that is, the output sequences of the inverters in the cascade, as well as the binary feedback sequence, that is, the input sequence to the first inverter in the cascade, are all phase shifts, possibly complemented, of each other, they are periodic with the same period if one of them, say, the feedback sequence, is periodic. The period of the vectorial state sequence, that is, the length of the corresponding cycle in the state-transition diagram is the least common multiple of the periods of r binary component state sequences and is, hence, equal to the period of the feedback sequence.

Let y denote the feedback sequence and let $(a_i)_{i=1}^r$ be the initial states of r delay elements which are as such the binary complements of the output values of the corresponding inverters, respectively. Then, the D-transform of y is given as

$$Y = \frac{1}{f(D)} \left(a(D) + \frac{b(D)}{1 + D} \right), \quad (5)$$

where the polynomial $a(D) = \sum_{i=0}^{r-1} D^i \sum_{j=i+1}^r f_j a_{j-i}$ characterizes the initial state and the polynomial

$b(D) = \sum_{i=0}^{r-1} D^i \sum_{j=i+1}^r f_j$ reflects the effect of inverters at the outputs of the delay elements, which are represented by adding in the all-one sequence whose D -transform is equal to $1/(1+D)$. As the degree of the polynomial $b(D) + (1+D)a(D)$ is at most r and the degree of $f(D)$ is r ($f_r = 1$), y has to be periodic for each initial state, which implies that the state-transition diagram consists of cycles only.

Further, note that the polynomial $b(D)$ satisfies $(1+D)b(D) = f(1) + f(D)$. If $f(1) = 0$, that is, if $f(D) = (1+D)h(D)$, then it follows that $b(D) = h(D)$ and, hence,

$$Y = \frac{h(D) + (1+D)a(D)}{(1+D^2)h(D)}. \quad (6)$$

Now, if $h(1) = 1$, then it is impossible that $1+D$ divides $h(D) + (1+D)a(D)$ (in particular, that $h(D) + (1+D)a(D) = 0$), as this would imply that $h(1) = 0$. As a consequence, the factor $1+D^2 = (1+D)^2$ has to be effectively present in the denominator, regardless of the initial state. Since $1+D$ is not a factor of $h(D)$, this further implies that the period of y has to be even, which means that all the cycles have even lengths and, in particular, that there are no fixed points, that is, cycles of length 1.

However, it is possible that $h(D) + (1+D)a(D)$ is equal to $h(D)$ and, hence, $Y = 1/(1+D^2)$, for $a(D) = 0$, i.e., for the all-zero initial state, or to $Dh(D)$ and, hence, $Y = D/(1+D^2)$, for $a(D) = h(D)$, i.e., for the all-one initial state. In both the cases, y is periodic with period 2 so that the all-zero and all-one states constitute a cycle of length 2.

For all other $2^r - 2$ initial states, the polynomial $h(D) + (1+D)a(D)$ contains neither $1+D$ nor $h(D)$ as a factor so that the cycle length is even and greater than 2. If $h(D)$ is an irreducible polynomial, then both $1+D^2$ and $h(D)$ have to be effectively present in the denominator and the period of y is then twice the exponent (period) of $h(D)$. In particular, if $h(D)$ is primitive, then the period of y is $2(2^{r-1} - 1) = 2^r - 2$, which implies that there is only one long cycle whose length is $2^r - 2$. $\square$

Proof of Theorem 3. The state vector consists of binary outputs of individual inverters in the cascade, taken in order of decreasing indexes, and the feedback value is the output of the inverter with index 1. We first characterize a Galois ring oscillator that has at least one fixed point. According to the corresponding state-transition function, a state vector with the feedback value equal to 0 is a fixed point if and only if r is even and it is equal to the alternating string $1010 \dots$. A state vector with the feedback value equal to 1 is a fixed point if and only if $f(1) = r \bmod 2$ (i.e., $1 + \sum_{i=1}^{r-1} (1 + f_i) = 1$) and it is equal to the bitwise XOR of $0, 1, 0, 1, \dots$ and $0, f_{r-1}, f_{r-2}, f_{r-3}, \dots, f_1$.

Consequently, a Galois ring oscillator does not have a fixed point if and only if neither of the two conditions given above are satisfied, that is, if and only if r is odd and $f(1) = 0$. $\square$

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on June 04, 2026 at 09:27:26 UTC from IEEE Xplore. Restrictions apply.

Proof of Theorem 4. The proof proceeds along similar lines as for Theorem 2. The first difference is in the proof that the period of the vectorial state sequence is equal to the period of the feedback sequence. Again, the period of the vectorial state sequence is the least common multiple of the periods of r binary component state sequences. The proof now follows from the fact that the feedback sequence is one of the binary component state sequences and that the periods of other binary component state sequences are equal to or divide the period of the feedback sequence due to the binary addition operation.

Equation (5) remains true, but, in this case, $a(D) = \sum_{i=0}^{r-1} D^i a_{r-i}$ and $b(D) = \sum_{i=0}^{r-1} D^i$. It follows that $b(D)$ satisfies $(1+D)b(D) = 1 + D^r$. Similarly, it follows that the state-transition diagram consists of cycles only, provided that the degree of $f(D)$ is r . If $f(1) = 0$, then (6) is used in the basic form

$$Y = (b(D) + (1+D)a(D)) / ((1+D^2)h(D)),$$

where $b(D) = \sum_{i=0}^{r-1} D^i$. Note that the condition that r is odd is equivalent to $b(1) = 1$. Consequently, if $b(1) = 1$, then $1+D$ cannot divide $b(D) + (1+D)a(D)$ so that the factor $1+D^2 = (1+D)^2$ has to be effectively present in the denominator, regardless of the initial state, and, hence, there are no fixed points.

Further, y is periodic with period 2 if and only if $b(D) + (1+D)a(D)$ is equal to $h(D)$ or $Dh(D)$. As $b(1) = 1$, this implies that $h(1) = 1$. If $h(1) = 1$, then there are exactly two initial states, that is, two polynomials $a(D)$ such that the period of y is equal to 2, namely, $a(D) = (b(D) + h(D)) / (1+D)$ and

$$a(D) = (b(D) + Dh(D)) / (1+D).$$

(The two initial states are specified after Theorem 4.) The rest of the proof is analogous to that of Theorem 2 with a remark that, if $h(D)$ is primitive and r is odd, then $r \geq 3$ and $h(1) = 1$. $\square$

ACKNOWLEDGMENTS

The author is grateful to Loris Bollea for implementing the experiments in FPGA technology and to the anonymous referees for useful suggestions. The methods are covered by two patent applications of Telecom Italia.

REFERENCES

- [1] B. Barak, R. Shaltiel, and E. Tromer, "True Random Number Generators Secure in a Changing Environment," *Proc. Conf. Cryptographic Hardware and Embedded Systems (CHES '03)*, pp. 166-180, 2003.
- [2] H. Bock, M. Bucci, and R. Luzzi, "Offset-Compensated Oscillator-Based Random Bit Source for Security Applications," *Proc. Conf. Cryptographic Hardware and Embedded Systems (CHES '04)*, pp. 268-281, 2004.
- [3] W.-T. Chuang and S.C. Hsu, "Enhanced Random Number Generator," US Patent No. US 6,240,432 B1, May 2001.
- [4] K.B. Coulthart, R.C. Fairfield, and R.L. Mortenson, "Random Number Generator," US Patent No. 4,641,102, Feb. 1987.

- [5] M. Dichtl, "How to Predict the Output of a Hardware Random Number Generator," *Proc. Conf. Cryptographic Hardware and Embedded Systems (CHES '03)*, pp. 181-188, 2003.
- [6] V. Fischer and M. Drutarovský, "True Random Number Generator Embedded in Reconfigurable Hardware," *Proc. Conf. Cryptographic Hardware and Embedded Systems (CHES '02)*, pp. 415-430, 2002.
- [7] G. Ghigo and L. Bollea, private communication, Telecom Italia Lab, Turin, 2004.
- [8] J.D. Golić, "Modes of Operation of Stream Ciphers," *Proc. Conf. Selected Areas in Cryptography (SAC '00)*, pp. 233-247, 2001.
- [9] J.D. Golić and R. Menicocci, "Statistical Distinguishers for Irregularly Decimated Linear Recurring Sequences," *IEEE Trans. Information Theory*, vol. 52, pp. 1153-1159, Mar. 2006.
- [10] M. Epstein, L. Hars, R. Krasinski, M. Rosner, and H. Zheng, "Design and Implementation of a True Random Number Generator Based on Digital Circuits Artifacts," *Proc. Conf. Cryptographic Hardware and Embedded Systems (CHES '03)*, pp. 152-165, 2003.
- [11] P.-Y. Liardet, "Random Number Generating Circuit and Process," US Patent No. US 6,581,078 B1, June 2003.
- [12] G. Marsaglia, "DIEHARD: A Battery of Tests of Randomness," 1996, <http://stat.fsu.edu/~geo>.
- [13] R.V.M. Oerlemans, "Digital True Random Number Generator Circuit," US Patent Application No. US 2002/0156819 A1, Oct. 2002.
- [14] T.E. Tkacik, "A Hardware Random Number Generator," *Proc. Conf. Cryptographic Hardware and Embedded Systems (CHES '02)*, pp. 450-453, 2002.
- [15] A. Juels, M. Jakobsson, E. Shriver, and B.K. Hillyer, "How to Turn Loaded Dice into Fair Coins," *IEEE Trans. Information Theory*, vol. 46, pp. 911-921, May 2000.



Jovan Dj. Golić received the BSc, MSc, and PhD degrees in electrical engineering from the School of Electrical Engineering, University of Belgrade, Belgrade, Yugoslavia, in 1979, 1981, and 1985, respectively. From 1979 to 1993, he worked at the Institute of Applied Mathematics and Electronics, Belgrade, where he was appointed a department head in 1986 and a senior research fellow in 1990. In 1987 and 1988, he was a Fulbright Visiting Scientist at the School of Electrical Engineering, Cornell University, Ithaca, New York. He was a part-time docent with the School of Electrical Engineering, University of Belgrade, from 1988 to 1997, as well as with the Faculty of Technical Sciences, University of Novi Sad, Novi Sad, Yugoslavia, from 1986 to 1997. Since 1985, he has been a part-time research associate at the Mathematical Institute, Serbian Academy of Science and Arts, Belgrade. From 1993 to 1997, he was a research scientist at the Information Security Research Centre, Queensland University of Technology, Brisbane, Australia. From 1997 to 2001, he worked as an associate professor with the School of Electrical Engineering, University of Belgrade. From 2001 to 2003, he was a chief cryptographer at the Rome CryptoDesign Center, Gemplus, Italy. In 2003, he joined the Telecom Italia Lab, Telecom Italia, Turin, Italy. He has taught and developed undergraduate and graduate courses in cryptology, information theory, data compression and error control coding, algebra, numerical analysis, and discrete mathematics. His research interests include design and cryptanalysis of stream and block ciphers, secure implementation of cryptosystems, sequences, coding and information theory, discrete mathematics, pattern recognition, and optimization techniques. He is a member of the IEEE and the International Association for Cryptologic Research.

► **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.**